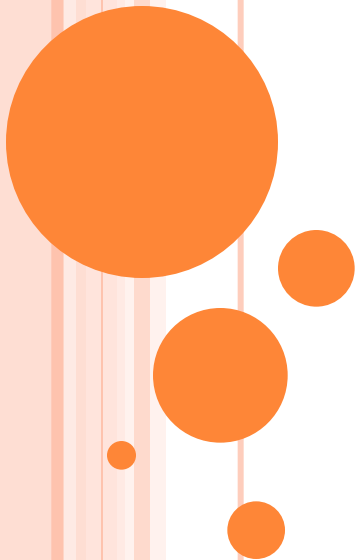


BASIC COMPUTER ORGANIZATION

Computer Organization and Assembly Language

Computer Science Department

National University of Computer and Emerging
Sciences Islamabad

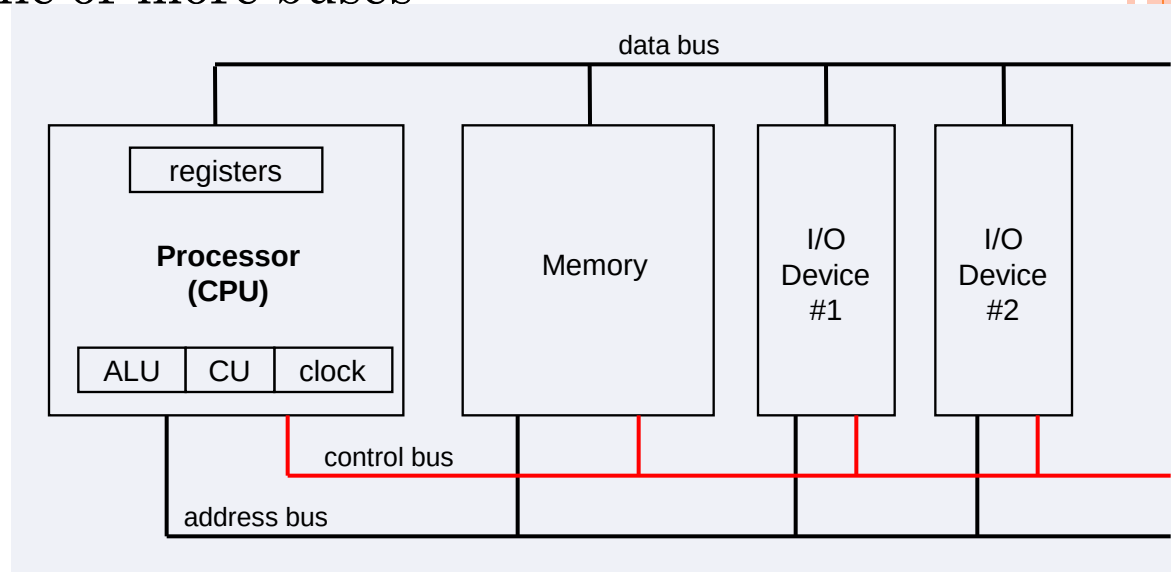


OVERVIEW

- Basic Computer Organization
- IA-32 Registers
- Intel Microprocessors
- Instruction Execution Cycle
- IA-32 Memory Management

BASIC COMPUTER ORGANIZATION

- Since the 1940's, computers have 3 classic components:
 - Processor, called also the CPU (Central Processing Unit)
 - Memory and Storage Devices
 - I/O Devices
- Interconnected with one or more buses
- Bus consists of
 - Data Bus
 - Address Bus
 - Control Bus



PROCESSOR

❑ Processor consists of

➤ Datapath

- ALU

- Registers

➤ Control unit

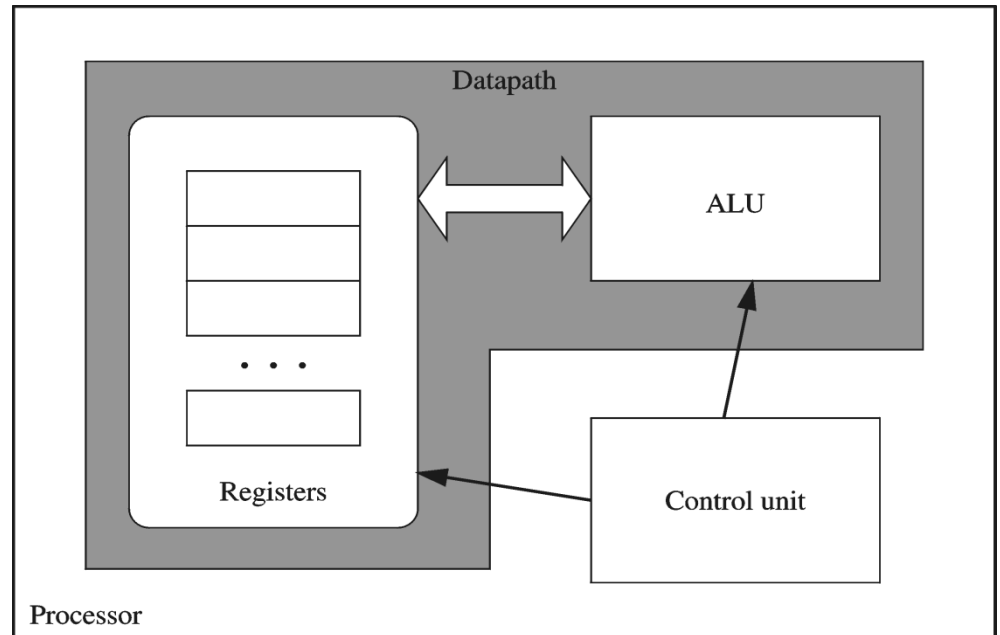
❑ ALU

➤ Performs arithmetic and logic instructions

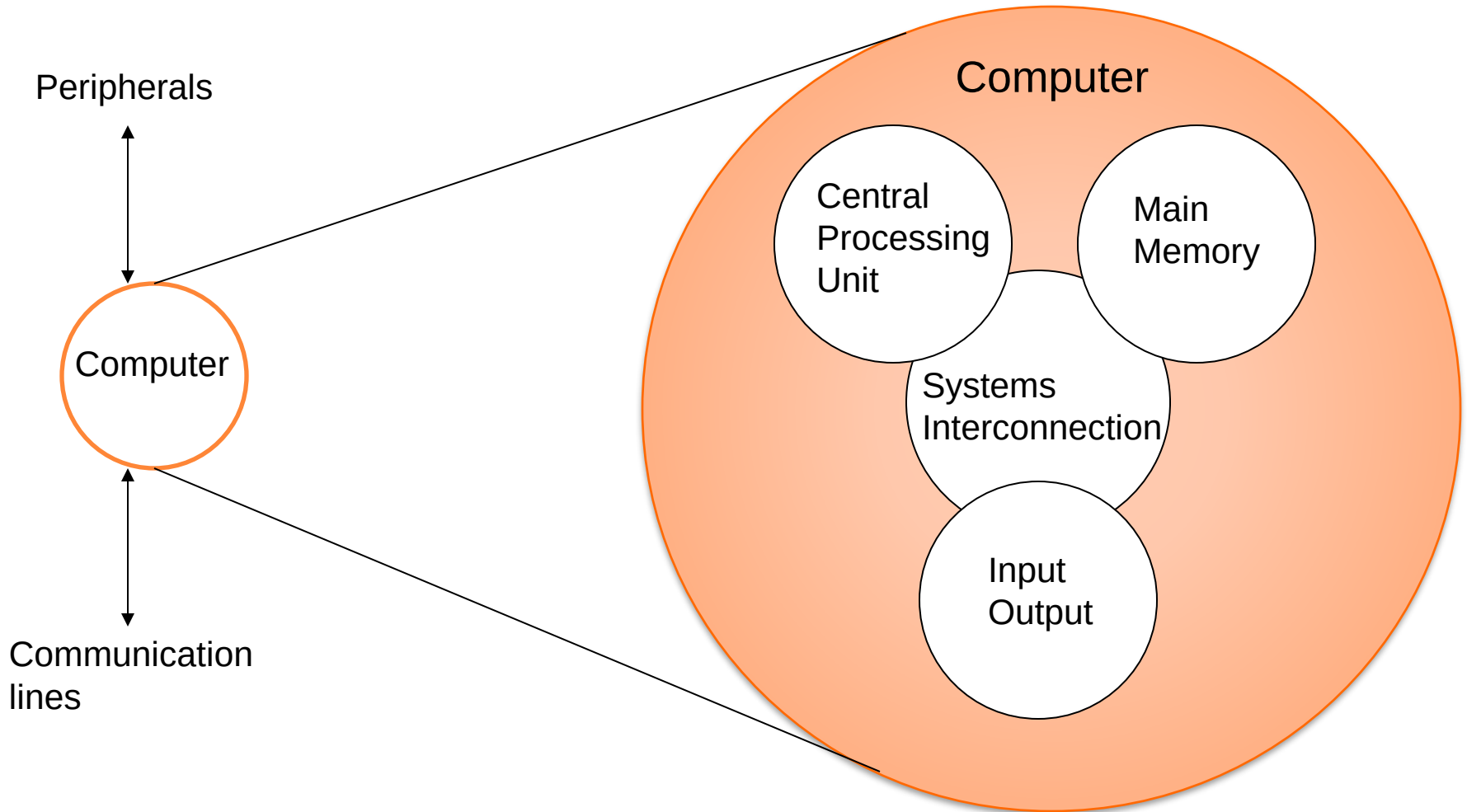
❑ Control unit (CU)

➤ Generates the control signals required to execute instructions

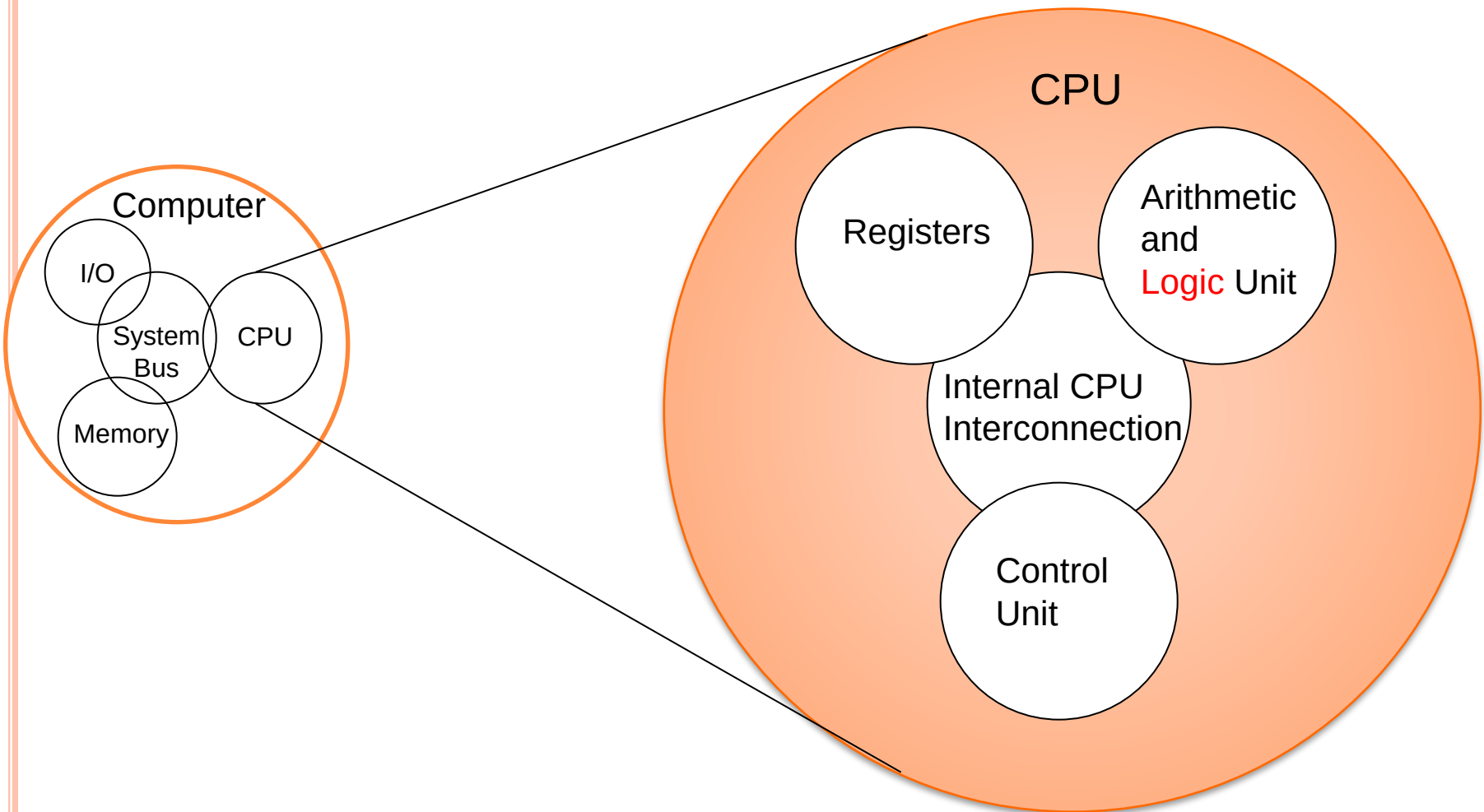
❑ Implementation varies from one processor to another



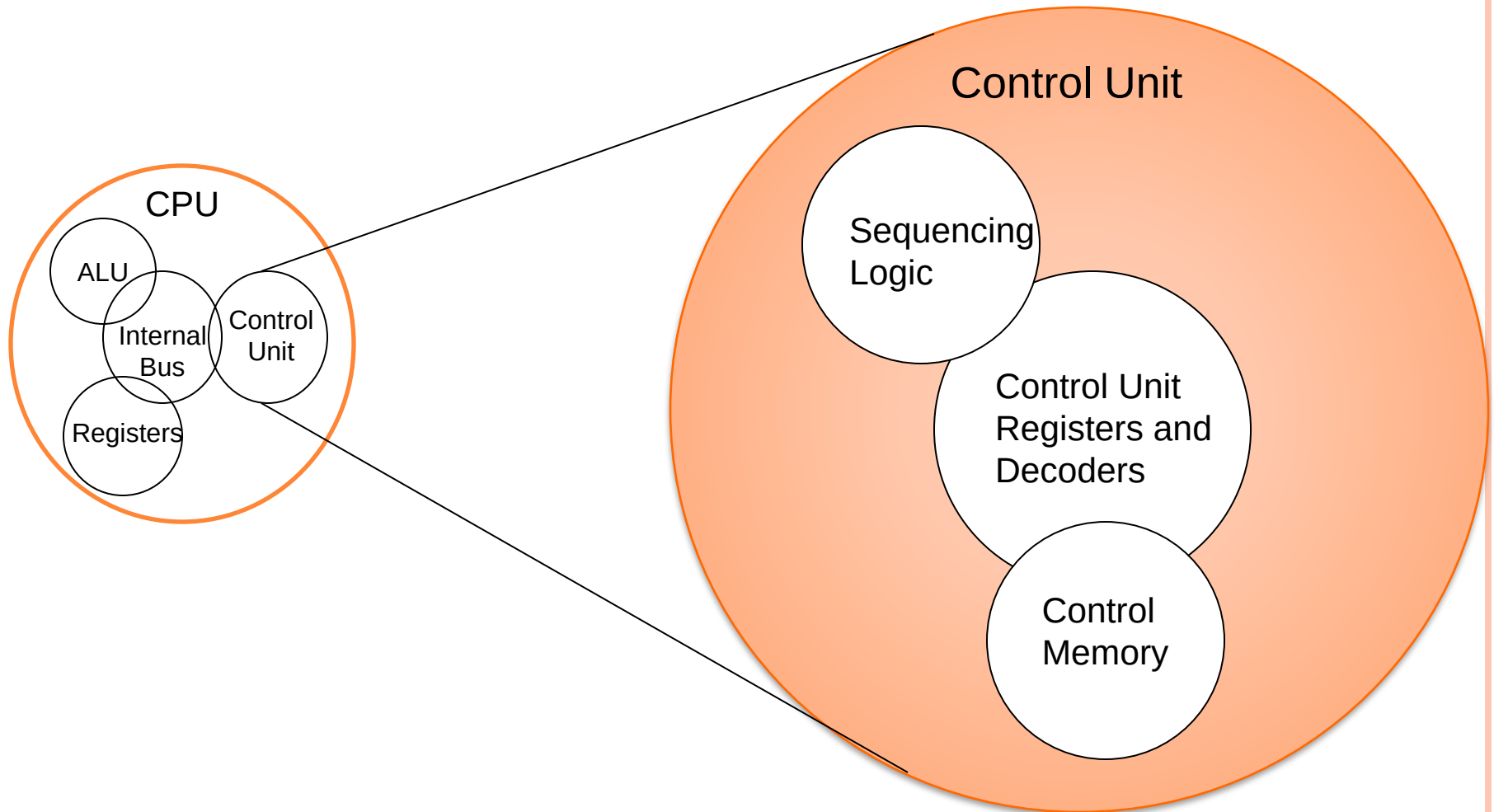
STRUCTURE - TOP LEVEL



STRUCTURE - THE CPU

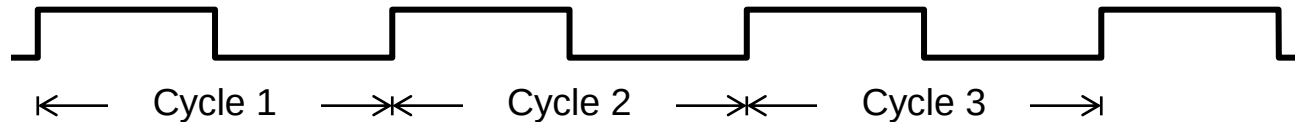


STRUCTURE - THE CONTROL UNIT



CLOCK

- Synchronizes Processor and Bus operations
- Clock cycle = Clock period = $1 / \text{Clock rate}$



- Clock rate = Clock frequency = Cycles per second
 - $1 \text{ Hz} = 1 \text{ cycle/sec}$ $1 \text{ KHz} = 10^3 \text{ cycles/sec}$
 - $1 \text{ MHz} = 10^6 \text{ cycles/sec}$ $1 \text{ GHz} = 10^9 \text{ cycles/sec}$
 - $2 \text{ GHz clock has a cycle time} = 1/(2 \times 10^9) = 0.5 \text{ nanosecond (ns)}$
- Clock cycles measure the execution of instructions

NEXT ...

- Basic Computer Organization
- IA-32 Registers
- Intel Microprocessors
- Instruction Execution Cycle
- IA-32 Memory Management

X86 PROCESSOR

386

786

5086

7086 => x86 Generation (x86 Assembly)

x86 Processor => 32 bits (4 bytes can be processed in parallel)

=> 64 bits (8 bytes can be processed parallel)

=> x86 (32bits/64 bits) Assembly

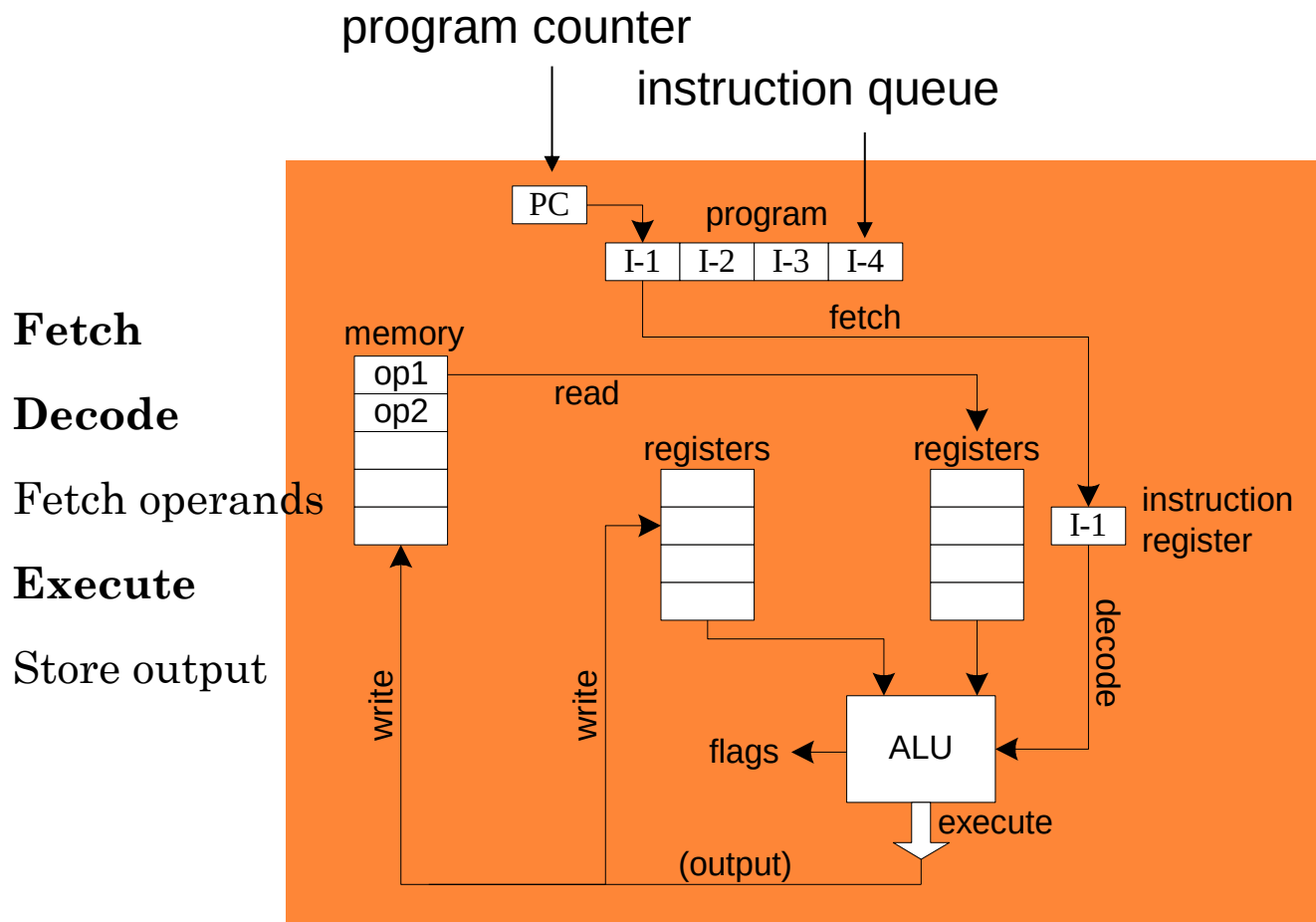
256 bits of source data=

64 bits architecture will take 4 cycles to complete processing.

32bits architecture will take 8 cycles.

INSTRUCTION EXECUTION CYCLE

- An instruction is not executed all at once, the CPU has to go through the *Instruction Execution Cycle* (a sequence of steps) to execute a machine instruction:
 1. CPU has to **fetch the instruction** from an area of memory called the *instruction queue*.
 2. Next, the CPU **decodes** the instruction by looking at its binary bit pattern.
 3. If operands are involved, the CPU **fetches the operands** from registers and memory.
 4. Next, the CPU **executes** the instruction, using any operand values it fetched during the earlier step.
 5. Finally, if an output operand was part of the instruction, the CPU **stores the result** of its execution in the operand.

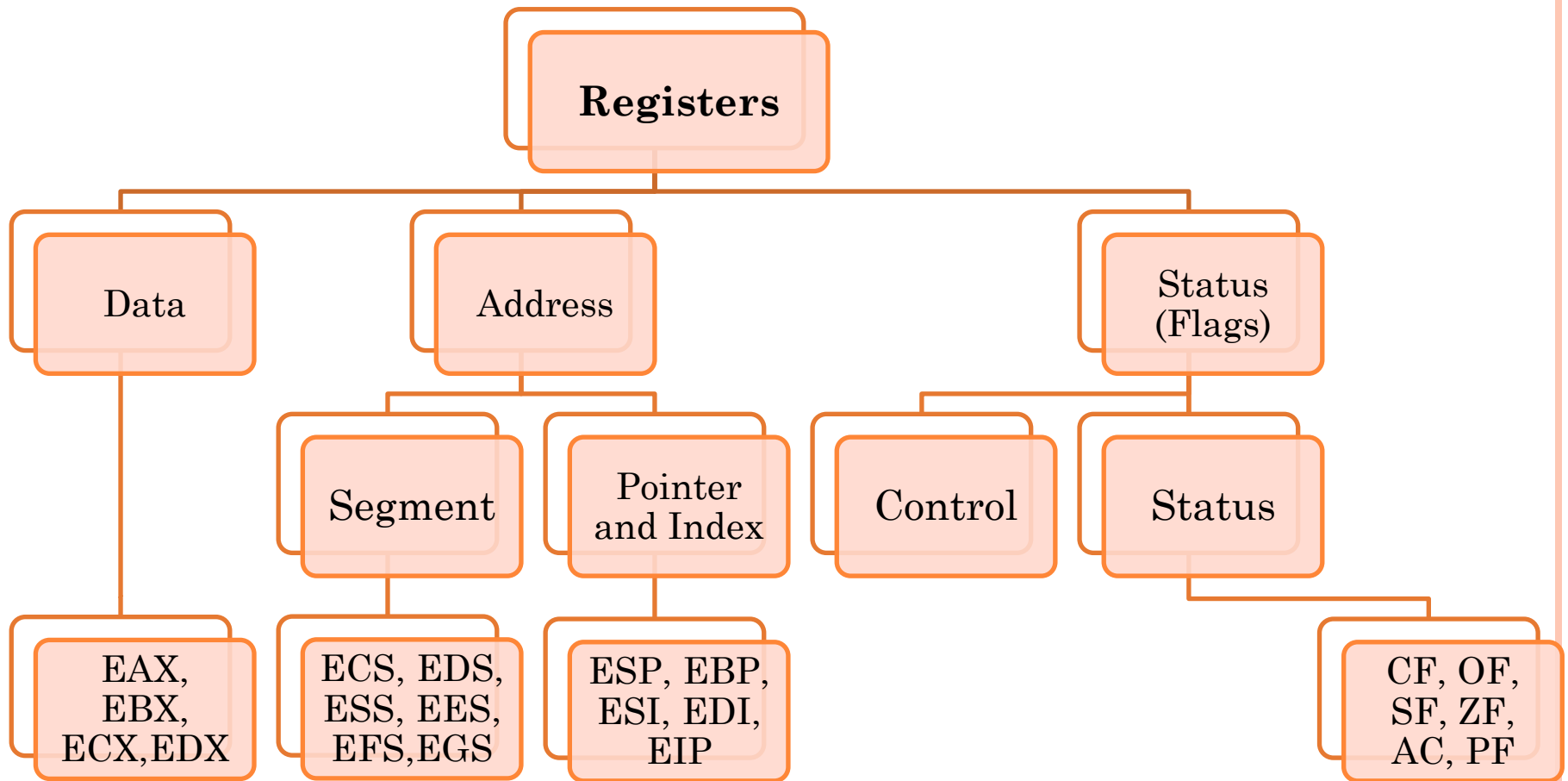


INSTRUCTION EXECUTION CYCLE

BASIC PROGRAM EXECUTION

REGISTERS

- **Registers** are high-speed named storage locations directly inside the CPU.
- The registers are classified according to the functions they perform:
 - **Data/General Purpose Registers** hold data for an operation
 - **Address registers** hold the address of an instruction or data
 - **Status register** keeps the current status of the processor.

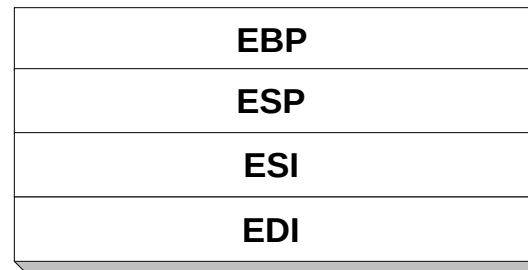
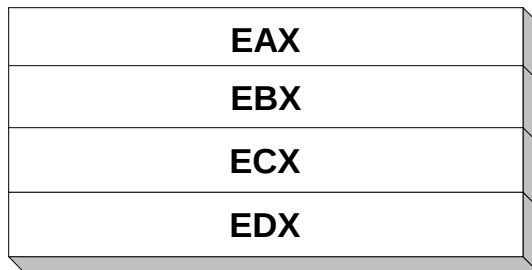


BASIC PROGRAM EXECUTION

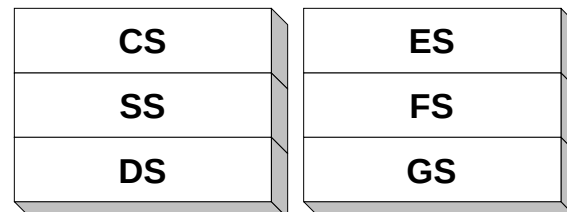
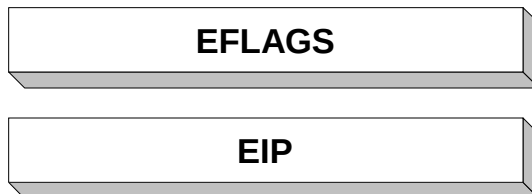
REGISTERS

- Registers are high speed memory inside the CPU
 - Eight 32-bit general-purpose registers
 - Six 16-bit segment registers
 - Processor Status Flags (EFLAGS) and Instruction Pointer (EIP)

32-bit General-Purpose Registers

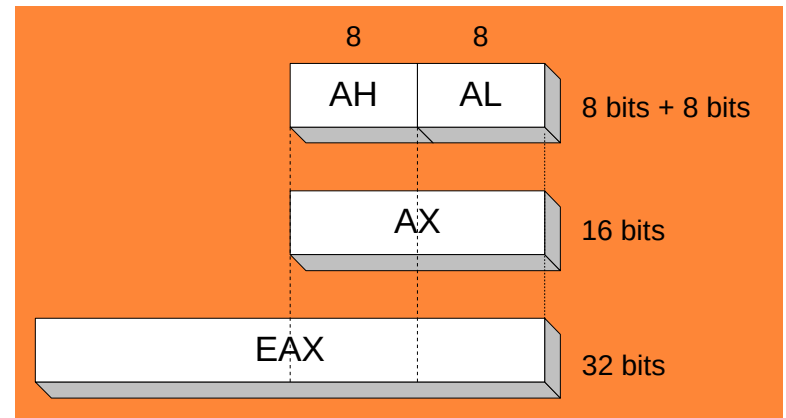


16-bit Segment Registers



ACCESSING PARTS OF REGISTERS

- EAX, EBX, ECX, and EDX are 32-bit **Extended** registers
 - Programmers can access their 16-bit and 8-bit parts
 - Lower 16-bit of EAX is named AX
 - AX is further divided into
 - AL = lower 8 bits
 - AH = upper 8 bits
- ESI, EDI, EBP, ESP have only 16-bit names for lower half



32-bit	16-bit	8-bit (high)	8-bit (low)
EAX	AX	AH	AL
EBX	BX	BH	BL
ECX	CX	CH	CL
EDX	DX	DH	DL

32-bit	16-bit
ESI	SI
EDI	DI
EBP	BP
ESP	SP

GENERAL-PURPOSE REGISTERS

- **Data Registers (EAX, EBX, ECX, EDX):** These four registers are available to the programmer for general data manipulation.
- **EAX** (Extended Accumulator Register) is preferred to use in arithmetic, logic and control instructions.
- **EBX** (Extended Base Register) is used to serve as an address register.
- **ECX** (Extended Counter Register) serves as loop counter.
- **EDX** (Extended Data Register) is used for input/output and also used with AX register for multiplication and division.
- The high and low bytes of the data registers can be accessed separately.

SEGMENTS

- **The 8086 uses a segmented memory model where memory is divided into segments, and addresses are formed by combining a segment address with an offset address'**
- SEGMENTS
- - Code
- - Data
- - Stack

MEMORY SEGMENT

- **Memory Segment:** A memory segment is a block of consecutive memory bytes. Each segment is identified by a segment number.
- Within a segment, a memory location is specified by giving an **offset**. This is the number of bytes from the beginning of the segment.
- A memory location may be specified by providing a segment number and an offset, written in the format **Segment:Offset**.
- E.g. **A4FB:4872h** means offset **4872h** within segment **A4FBh**.
- **Note:** The distance in bytes from the segment address to another location within the segment is expressed as an offset (or displacement).

Char A = {23, 24, 25, 26}
 Char B = {'A', 'S', 'D'}

I1
 I2: MOV AH, A[3];
 I3: **MOV AL, B[0];**

'A' => Base Address: 0000 00F0
 => Offset = 4 Bytes
 => Offset Address= Base Address
 + Offset
 0000 00F0 + 4 = **0000 00F4**

Data Segment

0000 00F0	23
0000 00F1	24
0000 00F2	25
0000 00F3	26
00F4	A
00F5	S
00F6	D

Base
 Address/
 Segment
 Address

 OFFSET

 OFFSET
 Address

Code Segment

F0F1 1000	I1
F0F1 1001	I2: MOV AH, A[3];
F0F1 1002	I3
.	I4
.	I5
.	I6
.	I7

ECS=F0F1 1000; EIP= F0F1 1001; EDS=0000 00F0 ESI= 0000 000F4

CODE, DATA AND STACK SEGMENT

- The program's code, data, and stack are loaded into different memory segments, we call them the **code segment**, **data segment**, and **stack segment**.
 - Stack segment holds local function variables and function parameters
- To keep track of the various program segments, segment register are used.
- The **ECS** (Extended Code Segment), **EDS** (Extended Data Segment), and **ESS** (Extended Stack Segment) registers contain the code, data, and stack segment numbers (base addresses) respectively.
 - If a program needs to access a second data segment, it can use the **EES** (Extended Extra segment) register.

POINTER AND INDEX REGISTER

- **Pointer and Index Register (ESP, EBP, ESI, EDI)**
- The registers ESP, EBP, ESI, and EDI normally point to (contain the offset addresses of) memory locations.
- Unlike segment registers, the pointers and index registers can be used in arithmetic and other operations.
- **ESP (Extended Stack Pointer):** this register is used in conjunction *with ESS* for accessing the stack segment.
- **EBP (Extended Base Pointer):** these registers are used to access data on the stack and other segments.
- **ESI (Extended Source Index):** these registers are used to point to memory locations in the data segment addressed by EDS.
- **EDI (Extended Destination Index):** these registers perform the same functions as ESI, these are used to access memory location addressed by EES.

INSTRUCTION POINTER (IP):

- **Extended Instruction Pointer (EIP):**
- The memory registers covered so far are for data access.
- To access instructions, x86 uses the ECS and EIP registers.
- The ECS contain the number(base address) of next instruction to be executed and EIP contains the offset.
 - EIP is updated every time an instruction is executed so that it will point to the next instruction.

GENERAL-PURPOSE REGISTERS

- Used primarily for arithmetic and data movement
 - **mov eax, 10** move constant 10 into register eax
- Specialized uses of Registers
 - EAX – Accumulator register
 - Automatically used by multiplication and division instructions
 - ECX – Counter register
 - Automatically used by LOOP instructions
 - ESP – Stack Pointer register
 - Used by PUSH and POP instructions, points to top of stack
 - ESI and EDI – Source Index and Destination Index register
 - Used by string instructions
 - EBP – Base Pointer register
 - Used to reference parameters and local variables on the stack

Base Address

- - Data Segment : held in Extended Data Segment **EDS**
- - Code Segment: held in Extended Code Segment **ECS**
- - Stack Segment: held in Extended Stack Segment **ESS**

Offset Address

- - Data Segment: held in **EBP, ESI, EDI**
- - Code Segment: held in **EIP**
- - Stack Segment: held in **ESP**

CS: IP
DS: BP
SS: SP
ES: SI/DI

• **FLAGS REGISTER**

- The purpose of the FLAGS register is to indicate the status of the microprocessor. It does this by the setting of individual bits called *flags*.
 - A flag is *set* when it equals 1; it is *clear* (or reset) when it equals 0.
- **General-purpose registers:** those can store both data and addresses
- **Status Registers** hold truth values often used to determine whether some instruction should or should not be executed

•FLAGS REGISTER

- **control flags** and **status flags**.
- The **control flags** enable or disable certain operations of the processor.
 - for example, if the IF (interrupt flag) is cleared (set to 0), inputs from the keyboard are ignored by the processor.
- The **status flags** reflect the result of an instruction executed by the processor.
 - For example, when a subtraction operation results in a 0, the **ZF** (zero flag) is set to 1 (true).

TYPES OF FLAGS



- 6 are the Status Flags
 - **CY, P, AC, Z, S, O**
- 3 are the Control Flags
 - **DF, IF, TF**
 - **How to remember Flags !!!! HINT !! (OD ITS ZAP C)**

TYPES OF FLAGS

- **Carry Flag**
- If any Arithmetic Operations generates a carry or a borrow out of the MSB (Most Significant Bit) of the result the Carry bit will SET (1). Otherwise will be RESET (0).
- **Example : 1111 + 1111**
 - **0000 - 1111**

TYPES OF FLAGS

○ Zero Flag

The Zero Flag is a condition flag in processors that is **set or reset based on the result of an operation**. The Zero Flag is typically set (i.e., becomes 1) if the result of an operation is zero and reset (i.e., becomes 0) if the result is non-zero.

Examples:

- $1111 - 1111$
- $\text{Add } 3 + (-3)$
- $10000000 + 10000000$

TYPES OF FLAGS

- **Sign Flag**
- The Sign Flag (S) is a condition flag in processors that **reflects the sign of the result of an operation**. It is usually set if the result is negative and reset if the result is non-negative (zero or positive).
- **Example:**
- **0101 – 0011**
- **0011 – 0101**
- **1111 - 1111**

TYPES OF FLAGS

- **Overflow Flag**

- The Overflow Flag (OF) is a condition flag used in CPU architectures to indicate when signed arithmetic operations result in an overflow. **Overflow occurs when the result of a signed operation exceeds the representable range for the given number of bits.**

TYPES OF FLAGS

- **Parity Flag:**
- The Parity Flag (PF) is a condition flag in CPU architectures that indicates **whether the number of set bits (1s) in the binary representation of the result of an operation is even or odd**. It is used for detecting errors in data transmission and other applications.
- **Example:**
- **01101101**
- **01101100**

TYPES OF FLAGS

- **Auxiliary Carry Flag**

- The Auxiliary Carry Flag (AF) is a condition flag used in CPU architectures, particularly in the context of binary-coded decimal (BCD) arithmetic. **The AF is set when there is a carry-out from the low-order nibble (4 bits) to the high-order nibble during operations like addition or subtraction.**

- **Example:**

- 0001 0101

- + 0000 1101

TYPES OF FLAGS

- **Interrupt Flag:**
- The Interrupt Flag (IF) is a condition flag in CPU architectures that controls the response of the processor to external interrupts. The Interrupt Flag is crucial for managing the interrupt mechanism and controlling whether interrupts can be processed.
- **IF (1): Ready to respond hardware request**
- **IF (0): Disable from responding the external hardware interrupt**

TYPES OF FLAGS

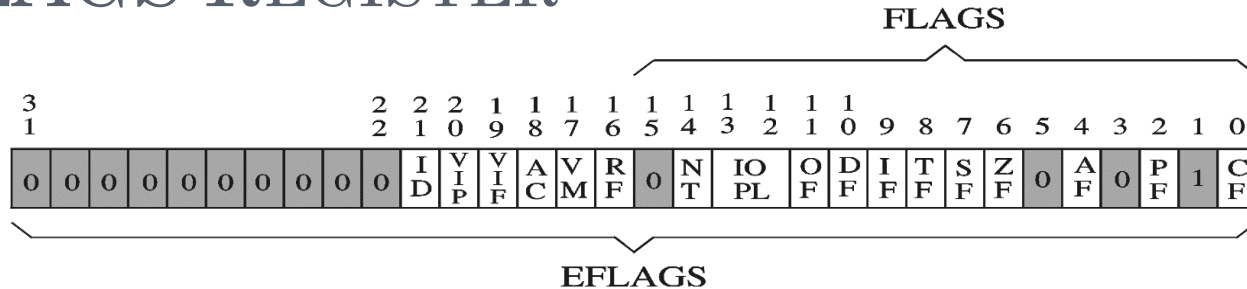
- **Direction Flag**

- The Direction Flag (DF) is a flag in CPU architectures that influences the direction of certain string operations, particularly in the x86 architecture. String operations involve moving or comparing blocks of memory (strings), and the **Direction Flag controls whether these operations proceed from higher memory addresses to lower ones (decrementing) or vice versa (incrementing)**

TYPES OF FLAGS

- **Trap Flag:**
- The Trap Flag (TF) is a condition flag in CPU architectures that, when set, causes the processor to enter **single-step mode during program execution**. Single-step mode allows the processor to execute one instruction at a time, facilitating debugging and analysis of code execution.

EFLAGS REGISTER



Status flags

CF = Carry flag

PF = Parity flag

AF = Auxiliary carry flag

ZF = Zero flag

SF = Sign flag

OF = Overflow flag

Control flags

DF = Direction flag

System flags

TF = Trap flag

IF = Interrupt flag

IOPL = I/O privilege level

NT = Nested task

RF = Resume flag

VM = Virtual 8086 mode

AC = Alignment check

VIF = Virtual interrupt flag

VIP = Virtual interrupt pending

ID = ID flag

❑ Status Flags

- Status of arithmetic and logical operations

❑ Control and System flags

- Control the CPU operation

❑ Programs can set and clear individual bits in the EFLAGS register

•FLAGS REGISTER

- **Carry Flag:** set when the result of an *unsigned* arithmetic operation is too large to fit into the destination.
 - **CF = 1** if there is a carry out from the most significant bit (msb) on addition, or there is a borrow into the MSB on subtraction.
- The **Overflow** flag (OF) is set when the result of a *signed* arithmetic operation is too large or too small to fit into the destination.
 - **OF = 1** if signed overflow occurred, otherwise it is 0

•FLAGS REGISTER

- The **Sign** flag (SF) is set when the result of an arithmetic or logical operation generates a negative result.
 - **SF = 1** if the msb of a result is 1; **SF = 0** if the msb is 0
- The **Zero** flag (ZF) is set when the result of an arithmetic or logical operation generates a result of zero.
- The **Auxiliary Carry** flag (AC) is set when an arithmetic operation causes a carry from bit 3 to bit 4 in an 8-bit operand.
- The **Parity** flag (PF) is set if the result contains an even number of 1 bits.

•FLAGS REGISTER

- In x86 processors, there are:
 - eight general-purpose registers
 - six segment registers
 - a processor status flags register (EFLAGS)
 - an instruction pointer (EIP).

STATUS FLAGS (SUMMARY)

- Carry Flag
 - Set when unsigned arithmetic result is out of range
- Overflow Flag
 - Set when signed arithmetic result is out of range
- Sign Flag
 - Copy of sign bit, set when result is negative
- Zero Flag
 - Set when result is zero
- Auxiliary Carry Flag
 - Set when there is a carry from bit 3 to bit 4
- Parity Flag
 - Set when parity is even
 - Sums the number of bits that are set in a number, and indicates whether the sum is odd or even

NEXT ...

- Basic Computer Organization
- IA-32 Registers
- Intel Microprocessors
- Instruction Execution Cycle
- IA-32 Memory Management

MEMORY

- Ordered sequence of bytes
 - The sequence number is called the memory address
- Byte addressable memory
 - Each byte has a unique address
 - Supported by almost all processors
- Physical address space
 - Determined by the address bus width
 - Pentium has a 32-bit address bus
 - Physical address space = **4GB = 2^{32} = 4000 000 000 bytes**
 - Itanium with a 64-bit address bus can support
 - Up to **2^{64} bytes** of physical address space

ADDRESS SPACE

Address (in decimal)		Address (in hex)
$2^{32}-1$		FFFFFFFF
		FFFFFFFE
		FFFFFFFD
	• • •	
2		00000002
1		00000001
0		00000000

Address Space is
the set of memory
locations (bytes)
that can be
addressed

MEMORY UNIT

□ Two Control Signals

○ Address Bus

- Address is placed on the address bus
- Address of location to be read/written

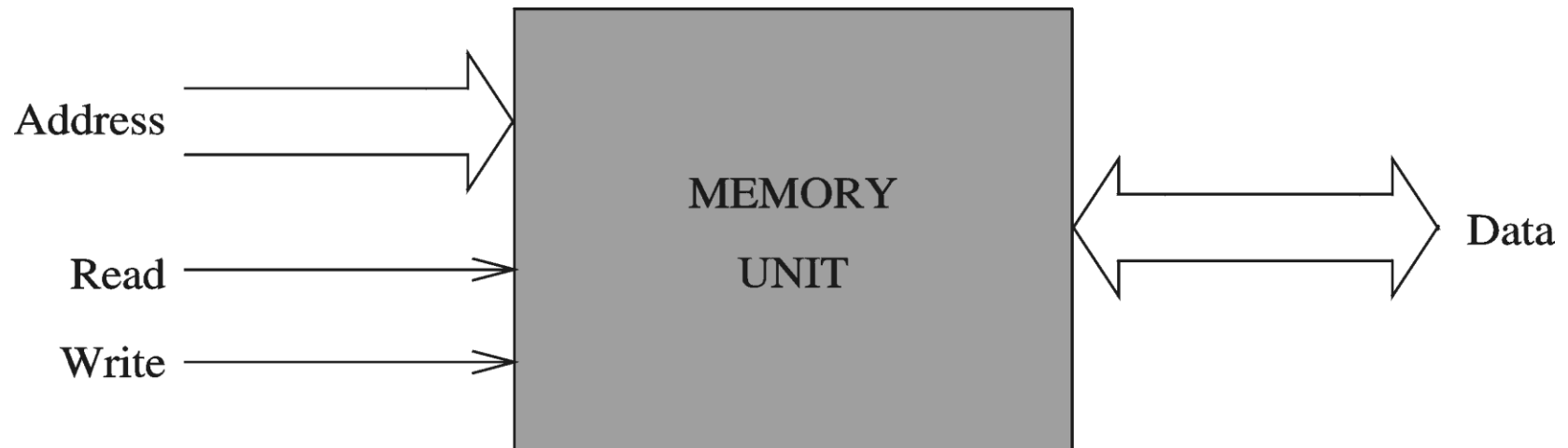
○ Data Bus

- Data is placed on the data bus

➤ Read

➤ Write

➤ Control whether memory should be read or written



MEMORY READ AND WRITE CYCLES

○ Read cycle

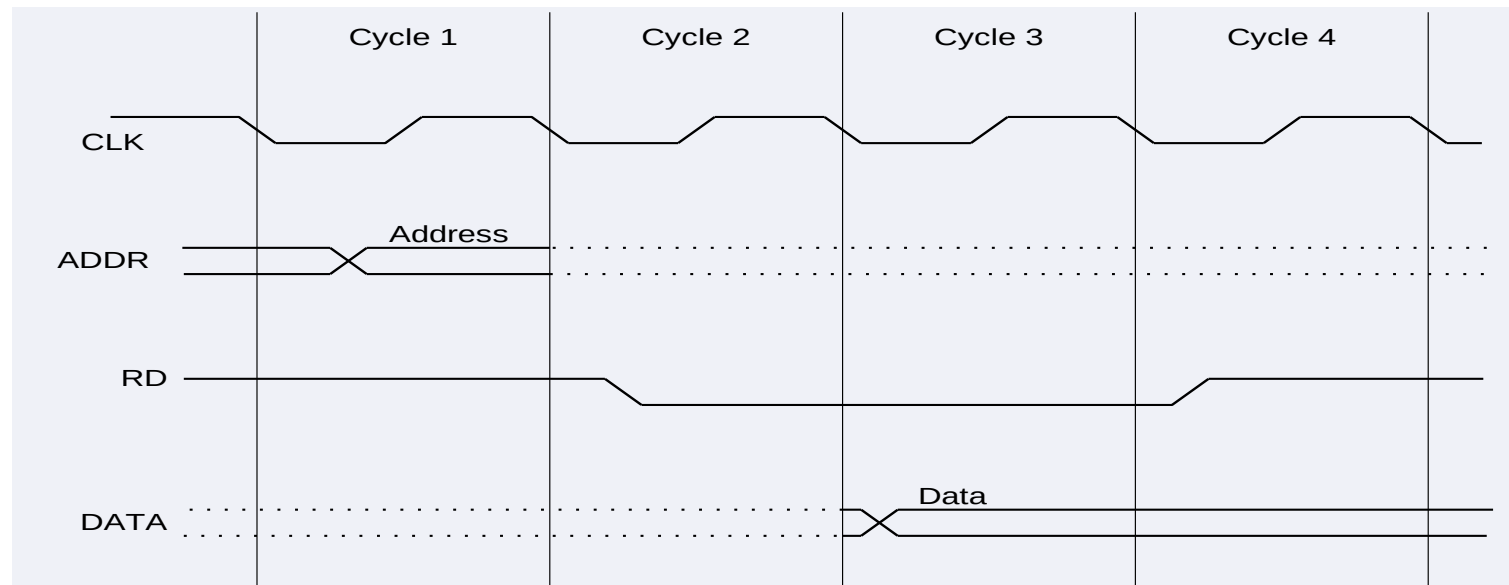
1. Processor places **address** on the address bus
2. Processor asserts the memory **read** control signal
3. Processor waits for memory to place the data on the data bus
4. Processor reads the **data** from the data bus
5. Processor drops the memory read signal

○ Write cycle

1. Processor places **address** on the address bus
2. Processor asserts the memory **write** control signal
3. Processor places the data on the data bus
4. Wait for memory to **store** the data (**wait states** for slow memory)
5. Processor drops the memory write signal

READING FROM MEMORY

- Multiple clock cycles are required
- Memory responds much more slowly than the CPU
 - Address is placed on address bus
 - Read Line (RD) goes low, indicating that processor wants to read
 - CPU waits (one or more cycles) for memory to respond
 - Read Line (RD) goes high, indicating that data is on the data bus



LITTLE ENDIAN ORDER

- Little endian order refers to the way Intel stores integers in memory.
- Multi-byte integers are stored in reverse order, with the least significant byte stored at the lowest address
- For example, the doubleword 12345678h would be stored as:

Byte	Offset
78	0000
56	0001
34	0002
12	0003

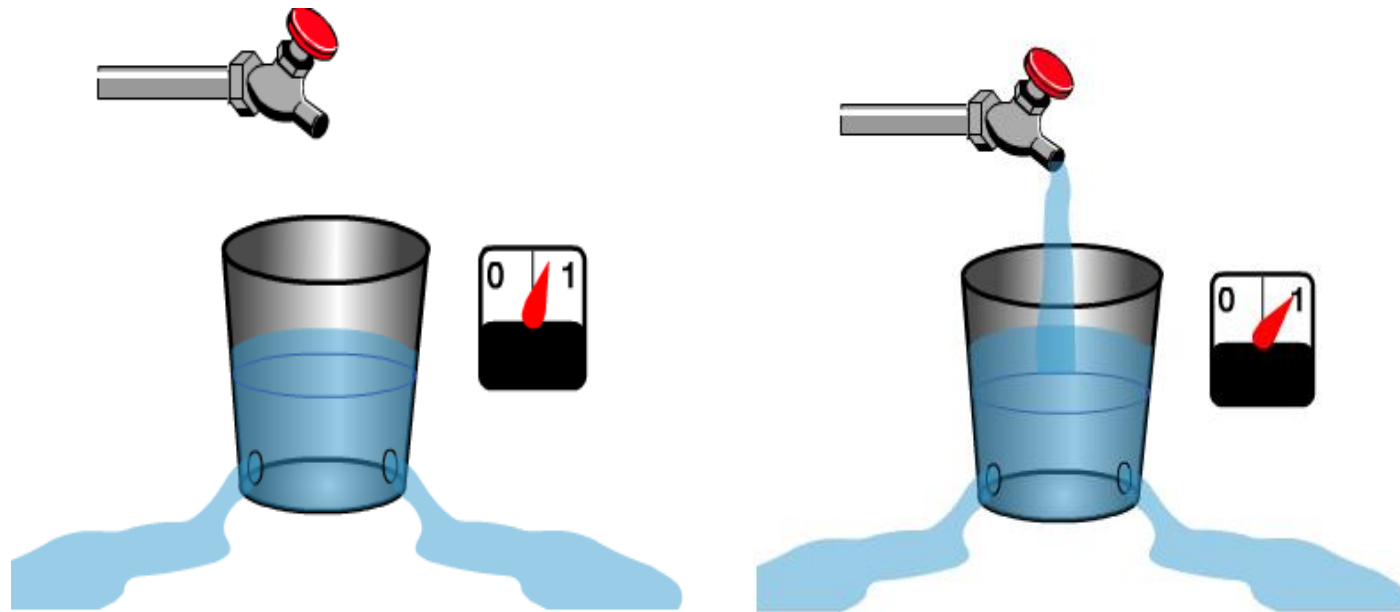
When integers are loaded from memory into registers, the bytes are automatically re-reversed into their correct positions.

BIG ENDIAN ORDER

- The Most significant byte stored at the lowest address
- For example, the doubleword 12345678h would be stored as:

Byte	Offset
12	0000
34	0001
56	0002
78	0003

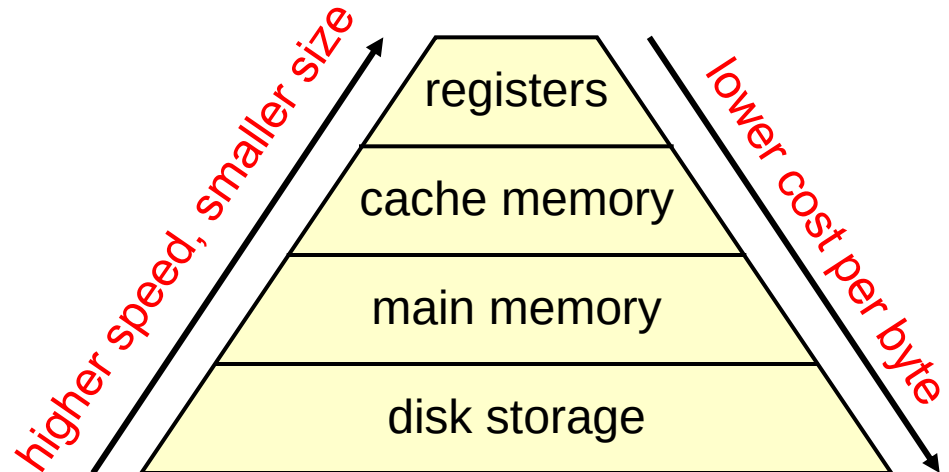
MOST RAM MUST BE RECHARGED CONSTANTLY



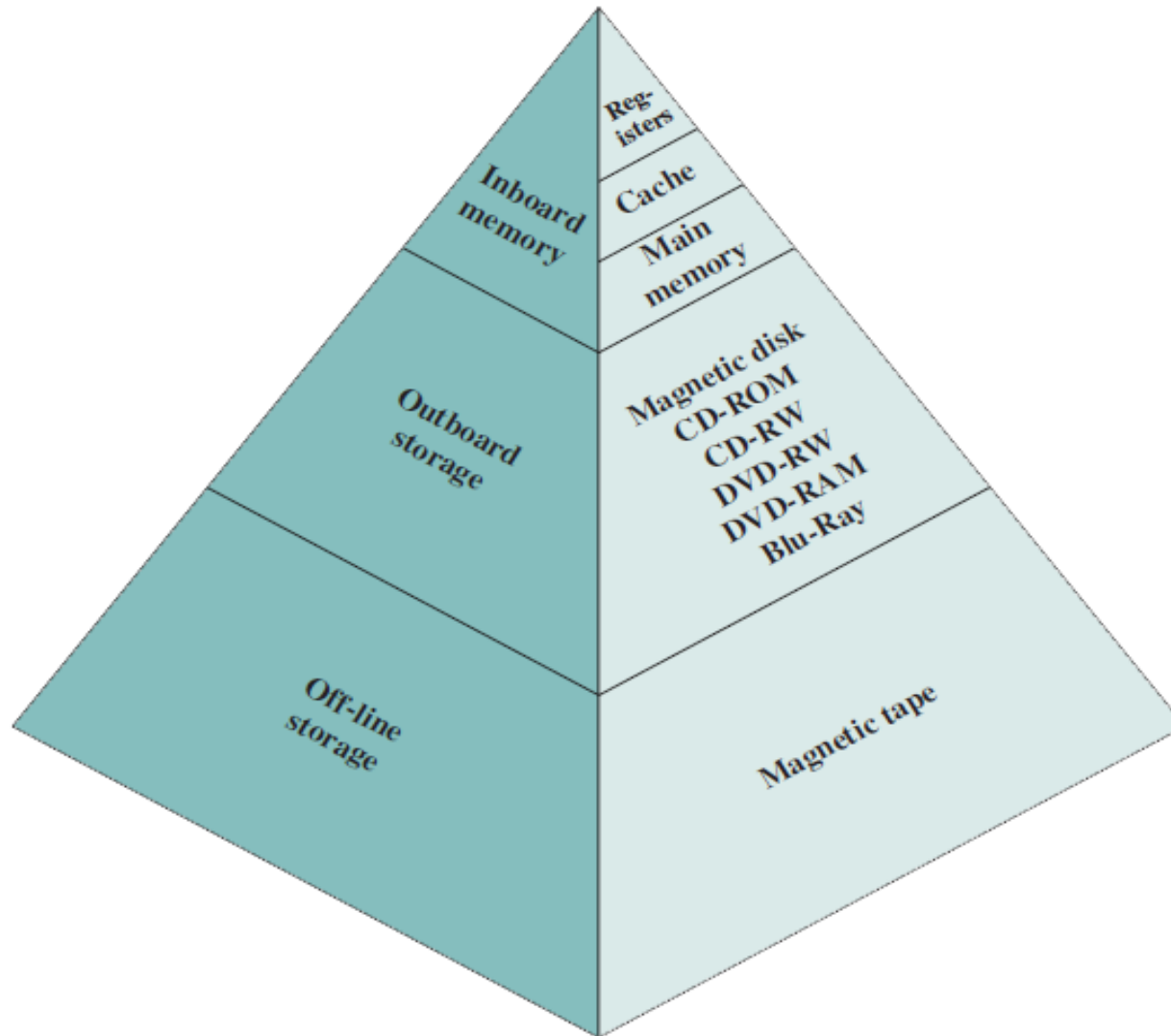
- Volatile Memory loses contents when power is removed

MEMORY HIERARCHY

- Registers
 - Fastest storage elements, stores most frequently used data
 - General-purpose registers: accessible to the programmer
 - Special-purpose registers: used internally by the microprocessor
- Cache Memory
 - Fast SRAM that stores recently used instructions and data
 - Recent processors have 2 and 3 levels
- Main Memory (DRAM)
- Disk Storage
 - Permanent magnetic
 - storage for files

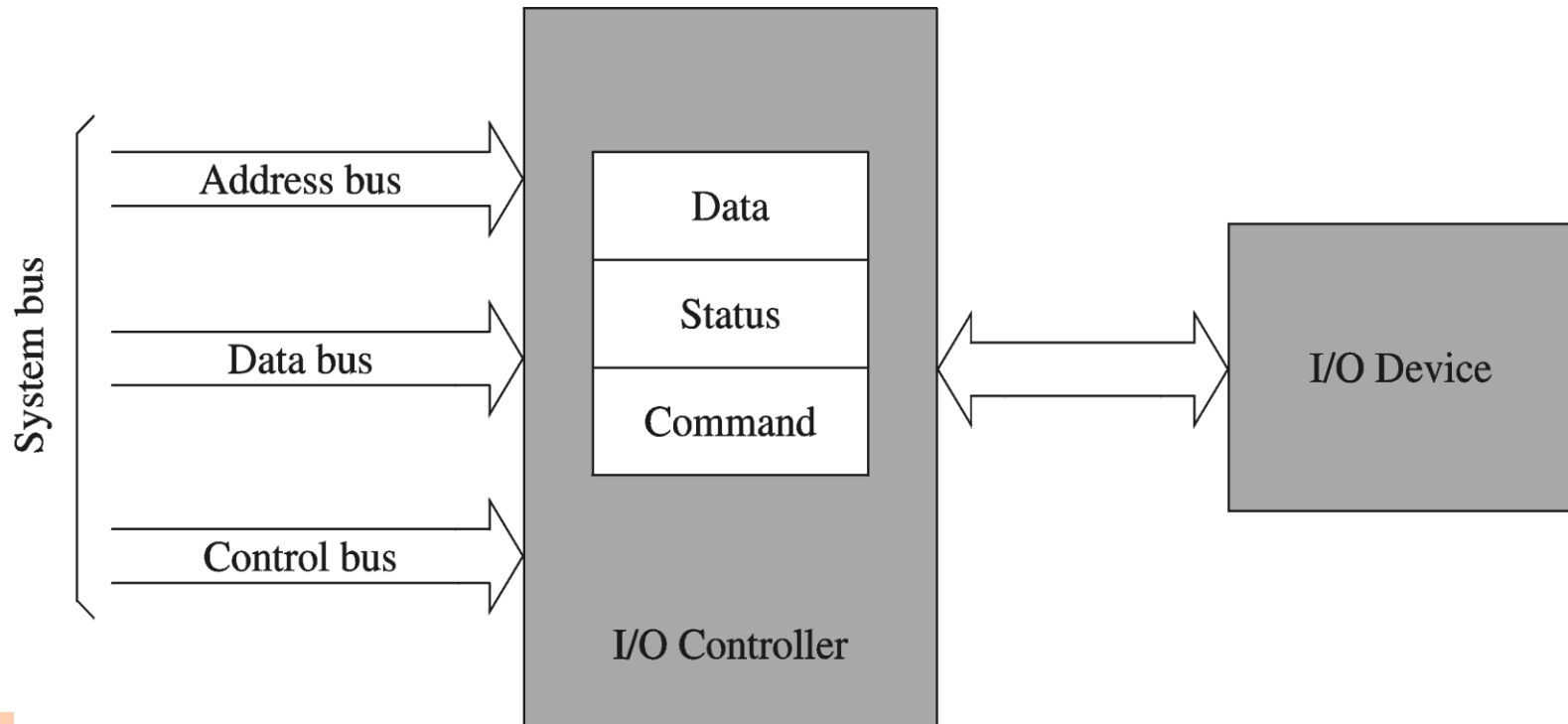


MEMORY HIERARCHY



I/O CONTROLLERS

- I/O devices are interfaced via an I/O controller
 - I/O controller uses the system bus to communicate with processor
 - I/O controller takes care of low-level operation details



REFERENCE

- Chapter # 1
 - Computer Organization and Architecture
 - William Stallings
- Chapter # 2
 - Assembly Language for x86 Processors
 - Kip R. Irvine